

Mathematical Definition of Connectivity

Student Project: Small-World Network Mapping

1 Connectivity is Intersection of Fan-in and Fan-out

Routing We have a Mixed-address routing scheme. A spike consists of the target core address and the source neuron address. The routers transport the spikes to the target cores (**Destination-Based Routing**). Once a spike reaches the target cores, the spike is broadcasted to all neurons. Based on the specified distance and the address of the source neuron, every neuron within the target core checks whether the spike comes from the slice it listens to (**Source-Based Routing**).

We observe that one can view the the Source-Based Routing and the Destination-Based Routing as two independent "filters". In order for a connection to exist on the chip, it needs to be both part of the Fan-out of the source neuron and part of the Fan-in of the target neuron. We define:

- Fan-in F_I : the set of connections specified by the source-address routing scheme (who the neurons listen to)
- Fan-out F_O : the set of connections specified by the destination-address routing scheme (which cores the neurons can send to)

The total set of connections C on the chip is the intersection of the Fan-in and the Fan-out:

$$C = F_I \cap F_O$$

The Fan-in and the Fan-out are hard **upper bounds** on the number of possible connections. The Fan-in is implemented by the look-up tables/ bit-masks that neurons use to filter out the incoming connections. The fan-out is flexible and can be configured arbitrarily. Moreover, we can "disable" individual neuron-neuron connections by setting the corresponding synaptic weight to zero. If we define F_O as the set of all possible edges, i.e.:

$$F_O = \bigcup_{i,j \in [n]} \{e_{ij}\},$$

then we obtain:

$$C = F_I.$$

2 Mapping Feasibility: Definition

Given the chip's parameter (the number of cores N , the number of neurons per core n , and the structure of the routing tree R), a graph $G = (V, E)$ is **(N, n, R)-feasible** if its nodes can be mapped to the neurons in the chip, so that every edge in the graph also exists on the chip, i.e.

$$E \subseteq F_I$$

Note: Unused Connections (i.e. connections on the chip that do not exist in the graph) are allowed. This is because by configuring the Fan-out and also by disabling individual neuron-to-neuron connections, we can "turn-off" superfluous, unused edges on the chip, also described in the matrix representation 3.2.

Note: Any graph becomes trivially feasible given a sufficiently large chip (Think about having one core per neuron or one core that fits all neurons).

3 Matrix Representation of the Connectivity

The matrix representation consists out of the Fan-In and Fan-Out matrices, multiplied by a hadamard product representing the intersection of both: $M = M_{\text{Fan-in}} \odot M_{\text{Fan-out}}$.

3.1 Matrix Representation of Fan-In

In this section we analyze the matrix representation of the **Fan-In**. Recall that the Fan-In is an upper bound on the possible connections on the chip, as described in Section 1.

We observe that the connectivity can be analyzed on two levels: On a Core Level and on a Neuron Level.

3.1.1 Core Level

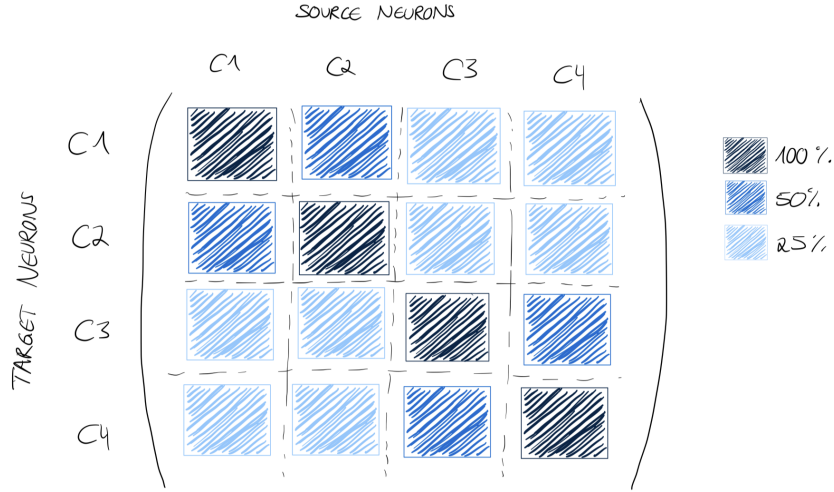


Figure 1:

When viewing the connectivity on the core level, we only consider the number of connections between cores, but not between individual neurons. In particular, the **neuron-level constraints** (neurons are divided into slices and listen to one slice per core, which is set globally for all cores with the same distances) are **not represented in this view**.

Figure 1 shows the connectivity structure between cores based on the example routing hierarchy shown in Figure 2. We observe the following properties:

1. **Exponential Decay:** The probability that neuron i is connected to j is $P(e_{ij} = 1) = \frac{1}{2^d}$ where d is the distance between the cores of neurons i and j .
2. **Symmetry:** We have $P(e_{ij} = 1) = P(e_{ji} = 1)$ (because the distance metric is symmetric).
3. **Recursion:** The pattern repeats itself, as illustrated by the example matrix shown in Fig. 5.

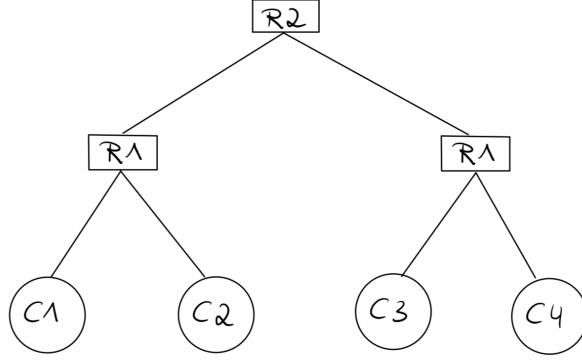


Figure 2: An example routing hierarchy.

As we can see, this connectivity structure can be modeled with a Hierarchical Stochastic Block Model, as further examined in Section 4.

3.1.2 Neuron Level

When viewing the connectivity on the neuron level, we must incorporate the neuron-level constraints (neurons are divided into slices and listen to one slice per core, which is set globally for all cores with the same distances).

Figure 3 shows an example connectivity between individual **neurons**. We observe the following properties and constraints:

1. For every row (target neuron): every neuron selects for every core C_i with distance d one of the **predefined slices** of size $\frac{1}{2^d}$ that it listens to. This slice needs to be the **same for all cores with same distance d** .
2. **Asymmetry**: We have no symmetry, since every neuron can configure the slice it listens to independently.

3.2 Matrix Representation of Fan-Out

Possibilities of the Fan-Out The **Fan-Out** is more flexible. A connection is enabled by the Fan-In when the synapse of the target neuron listens to the slice that includes the source neuron. Those enabled connections can still be disabled through two mechanisms.

- Configuring the Fan-out: The Source Neuron can decide to which cores it sends its spikes. Not addressing a core removes a connection *structurally*.
- Disabling Neuron-to-Neuron Connections: The weight w_{ij} of a specific connection between two neurons i, j can be set to 0, which *functionally* disables the connection.

Concrete Matrix Representation The Fan-In can be seen as an upper bound of the possible connections and the Fan-Out as an additional filter. Due to the two possibilities

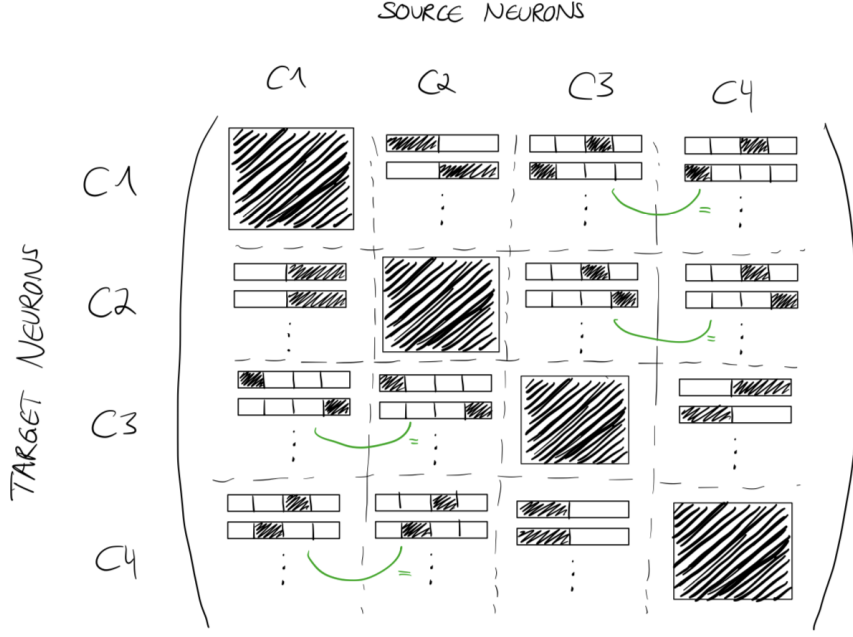


Figure 3: Example connectivity matrix for 4 cores and 2 routing levels. The green arrows indicate the blocks where the connectivity structure needs to be identical, since neurons can configure the fraction they listen to only globally for every distance.

for configuring the Fan-Out, we assume that the Fan-Out can arbitrarily filter out connections (either *functionally* or *structurally*). Therefore, the matrix representation of the Fan-out underlies to further constraints:

$$M_{\text{Fan-out}} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \in \{0, 1\}^{n \times n}$$

where n is the total number of neurons.

4 HSBM-based Algorithm

Insights

- The connectivity between cores on the chip fulfills the assumptions underlying the **Hierarchical Stochastic Block Model** (i.e. the connectivity between two cores only depends on their distance, which is equivalent to their Lowest Common Ancestor).
- Connectivity structure on Neuron and Core Level differ from each other (See Chapter 3.1). The SBM makes sense for the Core Level, but the constraints based on the slices are difficult to incorporate. Therefore, we propose a two step algorithm.

Assumptions

- **Hardware** parameters can be **flexibly adapted**, based on the parameters proposed by the algorithm. The HSBM struggles to incorporate hard hardware constraints, such as the maximum number of neurons per core. If the hardware parameters were fixed, the problem would turn into a strict assignment problem and the HSBM would be no longer suitable.

Simplifications

- Graph Generation in two separate stages: Neuron to Core Assignment, Neuron to Slice Assignment

Open question

- How do cluster assignment and slice assignment influence each other (possibly interleaved). The clustering and the core assignment also depend on the slice selection. This rigid 2-step approach may create neuron-core assignments which can then no longer be feasibly mapped to the chip.
- How to deal with the dependencies of slice selections between cores. In the ideal assumption, one could just assign the connections to neurons locally between two cores. That is not the case, because the slices are selected not per core but per distance (therefore it affects multiple cores). This is closely tied to the fact that it is difficult to split it into phases.

High-Level Algorithm

1. Assign nodes to clusters and construct routing tree (Core Level)
 - (a) Run statistical inference for Hierarchical SBM
Output: Router Hierarchy and assignment of neurons to cores.
2. Assign nodes to slices and compute bitmask for every neuron (Neuron Level):
Algorithm adapted from Vanessa Leite

- (a) Group neurons within the same core based on where they send their spikes to: if they send to the same neurons, put into the same slice
- (b) Clean up phase: check if by moving individual neurons we can further minimize inconsistencies

5 Non-Hierarchical Extended Stochastic Block Model and Inverse Inference

Insights

- The previous algorithm 4 does not necessarily provide correct results because of the rigid two-phase approach. There is the possibility that the resulting neuron-slice-assignment is valid afterwards and suffers from inconsistencies.
- One can look at the problem from two perspectives: to either map the graph perfectly and relax the hardware constraints (and potentially increase the hardware cost), or to relax the validity (and potentially increase the number of inconsistencies) and minimize the error while accommodating fixed hardware constraints.

Since the Neuron-to-Core assignment and the Neuron-to-Slice assignment are not independent, we need an approach which considers both assignments simultaneously. While alternative heuristics exist (e.g. a Matrix-based assignment algorithm or a greedy assignment algorithm, which selects a slice for a neuron and then assigns slices greedily), in this section we focus on a generative stochastic model approach.

High-level goal: Unify all constraints within a single statistical model, allowing us to use statistical inference for the final mapping.

5.1 Problem Formulation and Variable Definitions

We consider the problem of mapping an arbitrary directed network onto a **given and fixed** neuromorphic hardware HW . We approach the mapping problem by formulating a generative statistical model, adapting principles from the Degree-Corrected Stochastic Block Model. The idea is that we assume that the given input graph G was drawn from a parameterized (with parameter θ) distribution, where θ includes the configured assignments (neuron to core assignment, slice selection). Its density function: $p(x|\theta, HW)$ depend on the fix and known hardware and the configuration. The parameterized distributions has a density $\neq 0$ for all graphs, that comply with the Fan-In constraints. The flexible Fan-Out is modelled with random variables.

Note/ Future Idea: There would also be the possibility to incorporate the hardware as a random variable drawn from a prior, which would also enable us to regularize the hardware size for example.

Network Topology Let $G = (V, E)$ be a directed (as a first step unweighted) graph with $N = |V|$ nodes (neurons) and $m = |E|$ edges. The topology is represented by the adjacency matrix $A \in \{0, 1\}^{N \times N}$, where $A_{ij} = 1$ indicates a directed edge from node j to node i (node i receives a signal from node j). For each node i , we define its strictly positive observed degrees:

- $k_i^{in} = \sum_j A_{ij}$: The in-degree of node i .
- $k_j^{out} = \sum_i A_{ij}$: The out-degree of node j .

By definition, the total number of edges is $m = \sum_i k_i^{in} = \sum_j k_j^{out}$.

Hardware Architecture The hardware consists of C cores, each with a maximum capacity of n neurons such that $C \cdot n \geq N$. The cores are connected via a hierarchical router network.

This hierarchy defines a discrete distance function $d(c_i, c_j) \in \{0, 1, \dots, d_{max}\}$ between any two cores c_i and c_j . If $d(c_i, c_j) = 0$, the cores are identical and the internal network is fully connected.

For distances $d > 0$, the bandwidth is restricted. At a given distance d , the n addresses of a remote core are partitioned into S_d distinct, non-overlapping subsets called *slices*.

Let $B_{d,s} \subset \{1, \dots, n\}$ denote the set of local addresses belonging to slice index $s \in \{1, \dots, S_d\}$ at distance d . S_d grows exponentially with d : 2^d .

Mapping Variables The mapping of the network onto the hardware is defined by three sets of latent variables. The idea is that those are explicit parameters which we just do not know and want to infer:

- $c_i \in \{1, \dots, C\}$: The core to which node i is assigned.
- $x_i \in \{1, \dots, n\}$: The specific local address of node i within its assigned core.
- $s_{i,d} \in \{1, \dots, S_d\}$: The specific slice from which node i chooses to receive signals at distance d . Note that a node selects exactly one slice per hierarchical distance level $d \in \{1, \dots, d_{max}\}$.
- $\gamma_i^{in} > 0$: The *expected* in-degree of node i .
- $\gamma_j^{out} > 0$: The *expected* out-degree of node j .

One could also regularize them with a prior.

5.2 Hardware Constraints and Coupling Matrix

1. No two nodes can occupy the same local address on the same core. For all $i \neq j$, the mapping must satisfy: $(c_i = c_j) \implies (x_i \neq x_j)$
2. Each neuron has to be assigned exactly to one index and one core: $x_i, c_i, s_{i,d}$ are valid.

The Coupling Matrix W To represent whether a connection from node j to node i is hardware-compliant according to the **Fan-In**, we define the binary indicator function $\Delta_{ij} \in \{0, 1\}$. A connection is valid ($\Delta_{ij} = 1$) according to the **Fan-In** if the nodes are on the same core, or if the sending node j resides in the exact slice that the receiving node i has selected for their mutual core distance $d(c_i, c_j)$:

$$\Delta_{ij} = \begin{cases} 1, & \text{if } d(c_i, c_j) = 0 \\ 1, & \text{if } d(c_i, c_j) = d > 0 \text{ and } x_j \in B_{d,s_{i,d}} \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

Regarding the flexible **Fan-Out**, we need to model disabled connections. We employ α as the baseline probability for a compliant connection. Since we optimize for inconsistencies to the **Fan-In** here, we employ ϵ as the error/noise probability for a non-compliant connection. We define the coupling matrix $W \in \mathbb{R}^{N \times N}$ as $W_{ij} = \alpha \Delta_{ij} + \epsilon(1 - \Delta_{ij})$ where $\epsilon \ll \alpha$.

5.3 The Generative Stochastic Model

We model the network generation as a Poisson process. To ensure that the generated network can preserve a heterogeneous degree distribution and generates exactly m edges in expectation, we use a degree-corrected normalization.

Let λ_{ij} be the expected number of edges from node j to node i . We define λ_{ij} proportionally to the product of their respective expected degree parameters and the hardware coupling matrix W_{ij} :

$$\lambda_{ij} = m \frac{\gamma_i^{in} \gamma_j^{out} W_{ij}}{Z} \quad (2)$$

where Z is the normalization constant over the entire network, defined as the sum of all weighted expected degree products:

$$Z = \sum_{u=1}^N \sum_{v=1}^N \gamma_u^{in} \gamma_v^{out} W_{uv} \quad (3)$$

Proof of the global constraint: The expected total number of edges generated by this model is guaranteed to be m :

$$\mathbb{E} \left[\sum_{i,j} A_{ij} \right] = \sum_{i,j} \lambda_{ij} = \sum_{i,j} \left(m \frac{\gamma_i^{in} \gamma_j^{out} W_{ij}}{Z} \right) = \frac{m}{Z} \sum_{i,j} \gamma_i^{in} \gamma_j^{out} W_{ij} = \frac{m}{Z} Z = m \quad (4)$$

This intrinsic normalization ensures that the expected number of edges remains consistent regardless of the current mapping state or the parameters.

5.4 Likelihood Derivation and Inference Objective

Assuming the edges are drawn independently from the defined Poisson distribution, the likelihood of observing the specific adjacency matrix A given the full parameter set $\Theta = \{c, x, s, \gamma^{in}, \gamma^{out}\}$ is:

$$P(A | \Theta) = \prod_{i,j} \frac{\lambda_{ij}^{A_{ij}}}{A_{ij}!} e^{-\lambda_{ij}} \quad (5)$$

To facilitate statistical inference, we derive the log-likelihood $\mathcal{L}(\Theta) = \log P(A | \Theta)$. Dropping the constant term $\log(A_{ij}!)$ which depends solely on the observed data, we obtain:

$$\mathcal{L}(\Theta) = \sum_{i,j} (A_{ij} \log \lambda_{ij} - \lambda_{ij}) \quad (6)$$

The derivation has to be improved for some tricks. We tried to use the same approach

Substituting the definition of λ_{ij} and using the established global constraint $\sum_{i,j} \lambda_{ij} = m$, the expression simplifies:

$$\mathcal{L}(\Theta) = \sum_{i,j} A_{ij} \log \left(m \frac{\gamma_i^{in} \gamma_j^{out} W_{ij}}{Z} \right) - m \quad (7)$$

We expand the logarithm and use the topological definitions $\sum_j A_{ij} = k_i^{in}$, $\sum_i A_{ij} = k_j^{out}$, and $\sum_{i,j} A_{ij} = m$:

$$\mathcal{L}(\Theta) = m \log m - m - m \log Z + \sum_i k_i^{in} \log \gamma_i^{in} + \sum_j k_j^{out} \log \gamma_j^{out} + \sum_{i,j} A_{ij} \log W_{ij} \quad (8)$$

Dropping here the constant summand

$$\mathcal{L}(\Theta) \propto m \log Z + \sum_i k_i^{in} \log \gamma_i^{in} + \sum_j k_j^{out} \log \gamma_j^{out} + \sum_{i,j} A_{ij} \log W_{ij} \quad (9)$$

Note that this objective still is probably NP-hard and if we assume $\epsilon = 0$, we just translated the assignment problem into the statistical world, but the likelihood would just be 0 and 1.

A Higher Resolution Fan-In Matrix Representations

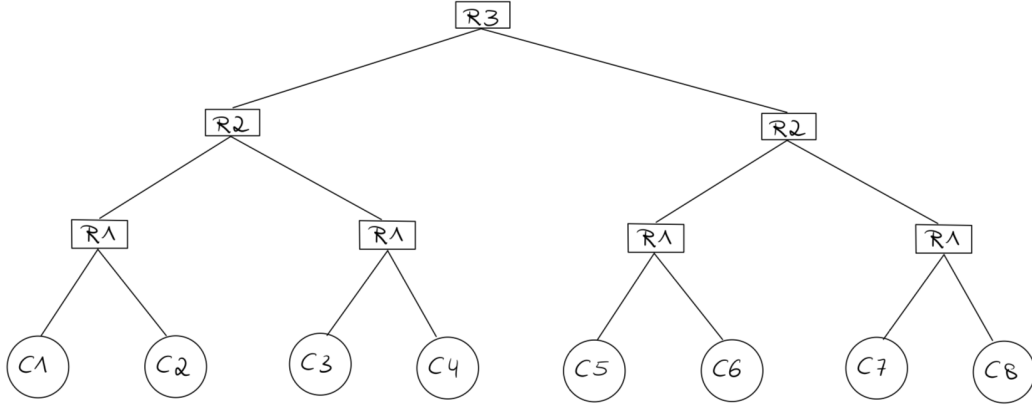


Figure 4: An example routing hierarchy with three levels.

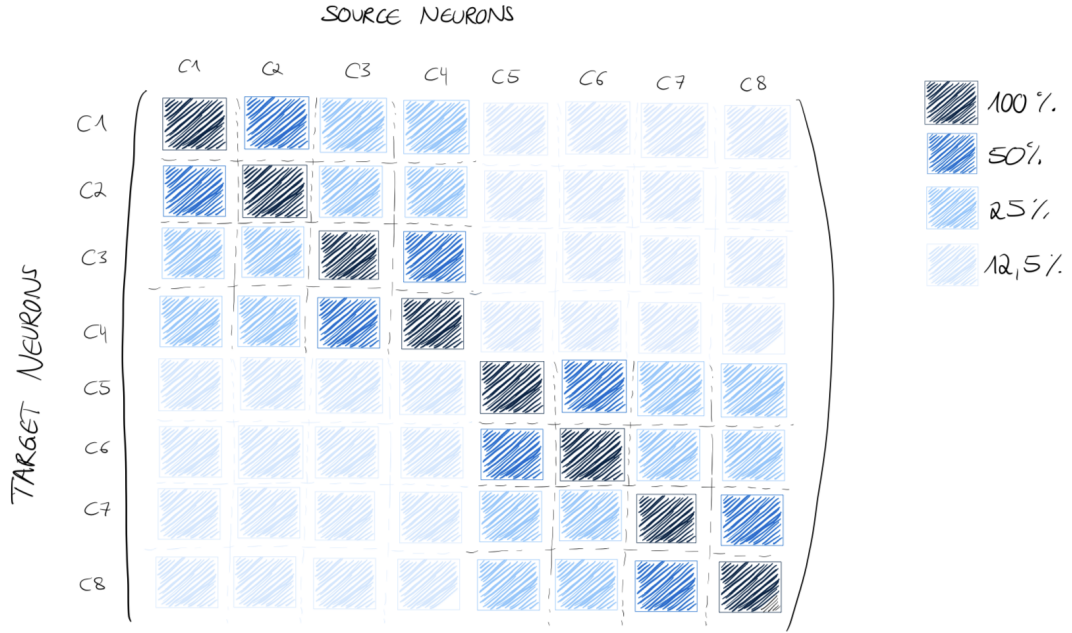


Figure 5: Core-wise connectivity matrix for 8 cores and 3 routing levels. The routing hierarchy is visualized in Fig. 4.

B Generative Fixed-Parameter Hierarchical Stochastic Block Model

Old idea that aimed to translate the hSBM, which produces variable-size clusters to fixed-size clusters. **The problem of the dependence between router hierarchy and slice assignment persists here**, therefore we do not pursue this model further.

Algorithm Generic Hardware HW is a structure defined by the following parameters:

- k : Neurons per core
- r : Number of children per router
- h : Height of the router tree / Also maximum possible distance between two cores

We assume that a given graph was generated as follows on the abstract level:

- k , r , and h were selected (Maybe regularization and selection based on Exponential model as well? $\rightarrow$ That would force the model to view smaller cores and smaller router-trees as more probable)
- Iterative layer-wise procedure: Assigns r cores/routers from previous levels'S router to a new router on the current level (**Same Question: Is it a random assignment or a selected assignment**). Samples random slice-indices for every neuron in the cluster of the new router, so for every group we get an Δ_{ij} , whether a connection is possible.
- Set a probability p (In the simple case without sampling of connection $p = 1$) and sample every connection between neurons i and j with probability $p \cdot \Delta_{ij}$.